Verificación de un servidor MCP para análisis PCA

Jorge Josué Flores González

Atlantis University

MAI 540: Machine Learning

Kevin A. García Gallardo

3 de noviembre de 2026

Verificación de un servidor MCP para análisis PCA

Se completó el servidor de análisis del profesor con dos herramientas, dos recursos y un prompt. Las cinco consultas respondieron correctamente en MCP Inspector, tanto por terminal como en su interfaz gráfica. Se conservaron registros y capturas, y se verificó que PCA produce los mismos resultados que las funciones iniciales para Iris y Wine con dos componentes.

El cliente solicita cargar_dataset para conocer las variables y luego ejecutar_pca para obtener varianza y cargas. Iris contiene 150 registros; sus dos componentes conservan el 95.81 % de la varianza. Los recursos proporcionan el catálogo y la ficha de Wine. El prompt devuelve instrucciones para interpretar resultados; esta verificación no genera una interpretación con un modelo de lenguaje.

La interfaz admite únicamente iris y wine, valida sus rutas y no ofrece escritura, comandos ni consultas de internet. No se incorporó el archivo .env. Estos controles limitan lo expuesto por el servidor, aunque no constituyen un aislamiento del proceso a nivel del sistema operativo.

MCP permite descubrir e invocar las mismas funciones desde clientes compatibles mediante un contrato común (Model Context Protocol, s. f.-b). Una importación directa resulta más simple dentro de Python, pero exige integrar cada cliente por separado. Aquí el protocolo facilita verificar la interfaz sin pagar una API.

El trabajo responde a la consigna de completar y verificar cinco piezas del servidor y documentar sus permisos (*Tarea 5.1*, 2026). La primera página de contenido resume la implementación, los resultados y el aporte del protocolo.

Flujo y evidencia de funcionamiento

Las verificaciones se realizaron con el Inspector por stdio y se repitieron en Safari tras iniciar mcp dev mcp_server.py en un proceso nuevo. Se introdujeron los argumentos y se comprobaron las respuestas en pantalla. Se utilizó MCP Inspector (Model Context Protocol, s. f.-a). Los registros JSON contienen los comandos y resultados; evidencia/capturas/ conserva las cinco capturas. No se utilizó una conversación con Claude.

1. cargar_dataset(nombre="iris") devolvió 150 filas, cuatro columnas numéricas y species como columna categórica. Esa descripción permite seleccionar variables antes del análisis.
2. ejecutar_pca(nombre="iris", n_componentes=2) devolvió proporciones de 0.7296 y 0.2285, acumulada de 0.9581 y cargas por variable. El resultado puede incorporarse a un informe o a una interpretación posterior.
3. data://datasets devolvió iris y wine. Este recurso permite consultar el catálogo sin ejecutar PCA.
4. data://datasets/wine devolvió 178 filas, 13 columnas numéricas y la columna categórica cultivar. La URI parametrizada identifica la ficha solicitada.
5. interpretar_componentes devolvió un mensaje con instrucciones para ejecutar PCA, revisar las cargas absolutas y valorar la varianza acumulada. Recuperar esta plantilla no equivale a ejecutar el análisis ni a validar una interpretación automática.

Cada pieza se incorporó en un commit separado. Las capturas muestran los resultados de las cinco consultas en la interfaz gráfica del Inspector 2.13.0. Se recortó únicamente la barra del navegador para excluir marcadores personales; el contenido de las respuestas permanece intacto.

Permisos y límites de la integración

El material inicial construía una ruta a partir del nombre recibido. Se añadió una lista de nombres autorizados y se comprobó que la ruta resuelta permanezca dentro de datasets/. Se rechazaron nombres desconocidos, rutas absolutas y recorridos con ../. La comparación con el código original confirmó que este ajuste conserva los resultados PCA de ambos datasets.

Solo se exponen metadatos y resultados agregados de los dos CSV autorizados. Credenciales, documentos personales y archivos de otras tareas no forman parte de las respuestas. .env, el entorno virtual y los archivos temporales quedan excluidos de Git. No deben agregarse datos sensibles a los CSV autorizados: la restricción de rutas no anonimiza su contenido.

El servidor no implementa operaciones de red ni escritura. El proceso conserva los permisos del usuario que lo ejecuta, por lo que esta garantía corresponde a las funciones expuestas y al código revisado, no a un bloqueo del sistema operativo. La instalación de dependencias requiere red; las consultas de análisis se comunican localmente mediante stdio.

El 3 de noviembre de 2026 se completó la comprobación gráfica desde un proceso nuevo: dos herramientas, dos recursos y un prompt respondieron correctamente. La conexión inicial demoró en listar las capacidades; una consulta repetida confirmó su disponibilidad. El código, el informe y las evidencias se conservan en https://github.com/Jorgefloresraiz/Tarea_5_1.

Referencias

Model Context Protocol. (s. f.-a). *MCP Inspector* [Software]. GitHub. https://github.com/modelcontextprotocol/inspector

Model Context Protocol. (s. f.-b). *MCP Python SDK* [Software]. GitHub. https://github.com/modelcontextprotocol/python-sdk

Tarea 5.1: Completar y verificar tu servidor MCP [Consigna de curso]. (2026). Atlantis University.